THE ARCHITECT'S FIELD NOTES

BEHAVIORAL INTERVIEW STUDY GUIDE / 07 OF 07

Design 5 — Real-Time Fraud Detection System

THINK CLEARLY. DECIDE WITH EVIDENCE. DELIVER WITH CONFIDENCE.

Evidence-based STAR answers, senior-level judgment, and focused interview rehearsal.

PREPARED FOR SENIOR / PRINCIPAL SOLUTION ARCHITECT INTERVIEWS

READ · REFLECT · PRACTICE

07



YOUR STUDY COMPANION

10-minute interview answer · American Express Solution Architect Preparation

NAVIGATION

Inside this guide

01 1. Problem & Requirements

02 2. High-Level Architecture

03 3. Key Design Decisions — With Reasons

04 4. End-to-End Request Flow

05 5. Data Model & Correctness

06 6. Failure Handling, Scalability & Security

07 7. Key Points to Remember

08 8. Interview Closing Summary

CHAPTER

1. Problem & Requirements

Design a system that detects potentially fraudulent credit card transactions **before authorization is approved**.

The system analyzes transaction patterns, applies fraud rules, and uses machine learning to generate a risk decision.

Functional requirements:

- Evaluate transactions in real time.
- Detect suspicious behavior using rules and ML.
- Track transaction velocity (frequency and amount).
- Return a risk decision and reason codes.
- Continuously update fraud signals using transaction events.

Non-functional requirements:

- **Low latency:** Must fit within the authorization response deadline.
- **High availability:** Fraud service failures must be handled safely.
- **Scalability:** Handle high transaction volumes.
- **Feature freshness:** Use recent transaction behavior.
- **Auditability:** Record why a decision was made.

Assumption: 10,000 peak TPS with an illustrative 100 ms p99 fraud-decision budget. These are design assumptions, not American Express figures.

CHAPTER

2. High-Level Architecture



```
      Kafka / Event Stream      |
           |                    |
           v                    |
      Stream Processor ----------+
           |
           v
    Historical Data Lake
           |
           v
     Model Training
```

Two distinct processing paths:

1. **Synchronous:** Evaluate the current transaction and return a risk decision.
2. **Asynchronous:** Process transaction events, update fraud features, and improve models.

The synchronous path must not wait for the Kafka pipeline to finish.

CHAPTER

3. Key Design Decisions — With Reasons

Decision	Choice	Why / Trade-off
Fraud evaluation	Synchronous API	Authorization needs an immediate risk decision
Detection	Rules + ML	Rules enforce explicit policies; ML detects complex patterns
Feature storage	Low-latency key-value/online feature store	Fast access to recent customer behavior
Streaming	Kafka + stream processor	Continuously update transaction features
Historical storage	Data lake/warehouse	Support training and investigations
Hard velocity limits	Atomic counters or coordinated state	Eventually updated counters may miss concurrent attempts
Reliability	Risk-approved fallback rules	Avoid uncontrolled fail-open behavior
Model deployment	Versioned serving with canary/shadow testing	Reduce risk from model changes
Scaling	Stateless scoring + partitioned event processing	Independently scale scoring and feature updates

Decision 1 — Why Rules and ML Together?

Rules engine: Evaluates explicit conditions.

Examples:

- Card reported stolen.
- Transaction violates a merchant restriction.
- Transaction exceeds a defined hard velocity limit.

ML model: Identifies combinations of suspicious signals.

Examples:

- Unusual purchase amount.
- Sudden spending pattern change.
- Suspicious device and merchant behavior.

Why both?

Rules provide deterministic controls. ML detects complex patterns that simple rules may miss.

Trade-off: ML needs training data, monitoring, explanations, and lifecycle management.

The final decision policy combines both outputs. A high ML risk score does not automatically override mandatory business rules.

Decision 2 — Why Use Kafka and Stream Processing?

Suppose a card experiences five purchase attempts within one minute.

The fraud service needs recent activity, not just historical averages.

```
Transaction Events
       |
       v
      Kafka
       |
       v
Stream Processor
       |
       v
Calculate:
- Attempts in last 1 minute
- Total amount in last 5 minutes
- Declines in last 10 minutes
       |
       v
Online Feature Store
```

Concept — Windowing: Grouping stream events over a defined time interval to calculate counts, totals, or other statistics.

Kafka Streams supports event-time processing, which distinguishes when an event occurred from when the processor received it. This matters for delayed and out-of-order events.

Why Kafka? It supports durable event ingestion, partitioned consumption, and replay.

Alternative: A managed stream service or another broker with equivalent capabilities.

Decision 3 — Why Use an Online Feature Store?

We cannot query a large historical warehouse for every authorization.

Instead, precompute frequently needed features.

Feature	Example
`txn_count_5m`	8
`total_amount_1h`	$2,500
`declined_count_10m`	4
`merchant_risk_score`	0.82
`avg_txn_amount_30d`	$75

Use a low-latency online store for scoring and a historical store for model training.

AWS SageMaker Feature Store, for example, distinguishes online low-latency feature access from offline historical feature storage.

Important limitation: Stream-updated features can lag behind the latest transactions.

If a rule requires an exact maximum of five attempts, an eventually updated stream counter alone is insufficient. Use coordinated atomic counting or another authoritative enforcement mechanism.

Remember: An online feature store accelerates decisions; it does not automatically guarantee fresh or strongly consistent data.

Decision 4 — Why Not Run ML Asynchronously?

Fraud detection must influence the authorization **before approval**.

If we approve first and run ML through Kafka afterward, we may detect fraud too late.

Therefore:

- Online risk scoring happens synchronously.
- Feature updates, model training, investigations, and reporting happen asynchronously.

This is the most important architectural distinction.

Decision 5 — How Do We Handle ML Failures?

Suppose the ML service times out.

Options:

Approach	Advantage	Disadvantage
Fail open	Fewer legitimate purchases interrupted	Higher fraud exposure
Fail closed	Stronger fraud protection	Higher false-decline risk
Degraded rules	Can retain limited safe decision-making	Less detection capability

Choice: A predefined, risk-approved degraded policy.

For example, use verified card information, available risk rules, and recent trusted features to evaluate transactions permitted under the fallback policy.

Mandatory controls must not be bypassed.

Use bounded timeouts, circuit breakers, and redundant scoring instances.

CHAPTER

4. End-to-End Request Flow

```
1. Authorization request arrives
               |
               v
2. Fraud Engine receives request
               |
               v
3. Fetch recent fraud features
```

```
                |
                v
  4. Evaluate mandatory rules
                |
                v
  5. Execute ML risk scoring
                |
                v
  6. Combine results using policy
                |
                v
  7. Return risk decision
                |
                v
  8. Authorization Service makes
     final approve/decline decision
                |
                v
  9. Publish transaction outcome
                |
                v
  10. Kafka updates features
      and historical records
```

Example risk decision:

```
{
  "transactionId": "txn_123",
  "riskScore": 0.91,
  "riskLevel": "HIGH",
  "recommendation": "DECLINE",
  "reasonCodes": ["HIGH_VELOCITY"],
  "modelVersion": "v12",
  "policyVersion": "p4"
}
```

The fraud recommendation is an input to authorization. The issuer applies the final policy, including any supported authentication or referral actions.

CHAPTER

5. Data Model & Correctness

Entity	Purpose
`TransactionEvent`	Authorization attempt and outcome
`FraudFeature`	Recent behavior and aggregated statistics
`RiskDecision`	Score, decision, reasons, and policy version
`ModelVersion`	Identifies the model used
`FraudCase`	Confirmed/suspected fraud and investigation outcome

Key correctness requirements:

- Duplicate transaction events must not inflate velocity counts.
- Hard limits need appropriate atomic enforcement.
- Fraud decisions must record model, rules, and feature freshness.
- Events arriving late must be handled according to defined windowing rules.
- Decisions must be auditable without unnecessarily exposing sensitive data.

CHAPTER

6. Failure Handling, Scalability & Security

Scenario	Design response	Why
ML service unavailable	Approved fallback policy	Controlled degradation
Feature store unavailable	Use permitted trusted features and fallback	Avoid uncontrolled decisions
Kafka unavailable	Durable event publication/outbox	Preserve updates for recovery
Duplicate events	Event-ID deduplication	Prevent inflated fraud counters
Out-of-order events	Event time, late-event handling	Correct historical aggregates
Traffic spike	Scale stateless fraud workers	Increase scoring capacity
Model accuracy degrades	Monitor drift, rollback model	Reduce incorrect decisions
Sensitive data exposure	Tokenization, encryption, restricted access	Protect financial/customer data

Scaling Strategy

- **Fraud API:** Horizontally scale stateless instances.
- **ML serving:** Scale inference instances independently.
- **Kafka:** Partition account-level events using stable account keys.
- **Feature store:** Distribute key-based access and monitor hotspots.
- **Data lake:** Store historical events separately from online scoring.

Operational Metrics

Monitor p99 scoring latency, feature age, service errors, model drift, fraud detection rate, false-decline rate, and financial losses.

For ML governance, NIST's AI Risk Management Framework emphasizes ongoing measurement, monitoring, and management of AI risks.

CHAPTER

7. Key Points to Remember

1. **Synchronous scoring, asynchronous updates.** Fraud decisions must happen before approval; event streams update historical and recent signals independently.
2. **Rules + ML.** Rules enforce known policies; ML identifies complex fraud patterns.
3. **Online feature store for low latency.** Precompute recent behavioral features rather than querying historical data for every authorization.
4. **Feature freshness matters.** Stream processing may lag. Hard velocity limits require stronger coordination.

5. **Fraud protection must fail safely.** Use explicit fallback rules, model versioning, monitoring, and auditability.

CHAPTER

8. Interview Closing Summary

> I would design fraud detection with a synchronous risk engine using business rules, ML models, and a low-latency online feature store. Kafka and stream processing continuously update behavioral features, while historical data supports model training and investigations. Mandatory rules and hard velocity constraints use appropriately consistent state, because asynchronous features may lag. I would scale scoring independently, implement risk-approved fallback policies, and monitor model performance, false declines, latency, and feature freshness. This balances fraud prevention, customer experience, and system reliability.

Next design: Payment Gateway & Orchestration System — routing across payment processors, external API failures, retries, circuit breakers, payment state machines, and preventing duplicate charges across providers.